

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

FPT UNIVERSITY

MASTER OF SOFTWARE ENGINEERING (MSE)
GRADUATE THESIS PROPOSAL

Thesis Title:

**AN EDGE-CLOUD
COLLABORATIVE IoT
FRAMEWORK
FOR REAL-TIME SMART
SURVEILLANCE**

Integrating Dynamic Projection People Counting,
Cosine Face Identification, and Multi-Hazard
Anomaly Detection

Academic Supervisor: Dr. Doan Xuan Huy Minh
Graduate Student: Nguyen Van Toan
Student ID / Class: MSE26HCM

Ho Chi Minh City, May 2026

Abstract

In contemporary security management, the demand for smart, real-time, and privacy-preserving surveillance systems has grown exponentially. Commercial off-the-shelf cameras often rely heavily on cloud-based computation, introducing significant network latency, bandwidth overheads, and severe data privacy concerns. This capstone project proposal presents a comprehensive, lightweight, and low-cost IoT-Edge-Cloud collaborative framework designed for local real-time intelligent monitoring.

The framework integrates four key functional capabilities deployed across an Edge layer and a Central Web Application: (1) an optimized multi-camera people counting model utilizing YOLOv8s and ByteTrack under a custom geometric double-line projection state machine with crossing cooldowns and dynamic state-inheritance; (2) a real-time face verification module leveraging Multitask Cascaded Convolutional Networks (MTCNN) for alignment, FaceNet for feature embedding generation, and cosine distance metric classification; (3) a Region-of-Interest (ROI) intrusion detection module utilizing YOLOv5; and (4) an automated fall detection system utilizing custom trained YOLOv8-fall weights.

To prevent network blocking and streaming lag under real-time alerts, we implement an asynchronous multi-threaded notification dispatcher connecting directly to Telegram services. Preliminary experiments show that the system achieves high accuracy in crowd tracking, eliminates counting jitter near boundaries, and operates at ≥ 20 FPS on a CUDA-accelerated edge GPU with latency below 1.5 seconds. This proposed system establishes a robust, highly extensible architecture suitable for household and office automation.

Contents

Abstract	1
List of Abbreviations	4
1 Introduction	5
1.1 Background and Motivation	5
1.2 Problem Statement	5
1.3 Thesis Objectives	6
1.4 Project Scope	6
1.5 Organization of the Thesis	6
2 Literature Review	8
2.1 Edge-Cloud IoT Frameworks in Video Surveillance	8
2.2 Object Detection and Multi-Object Tracking (MOT)	8
2.3 Face Verification and Embedding Extraction	8
2.4 Visual Anomaly Detection: Fall and Fire Hazards	8
2.5 State Machines in Spatial Crossing Analysis	8
3 Design and Implementation	9
3.1 Overall System Architecture and Data Flow	9
3.2 Projection-based Double-Line Crossing Model	9
3.2.1 State Transitions and Cooldown Filtering	10
3.3 Dynamic Lost-Track State Inheritance	10
3.3.1 Novelty Validation Case Study	12
3.4 Face Recognition Pipeline with Profile/Tilted Adaptation	12
3.5 Multi-Hazard Anomaly Detection and H.264 Video Buffering	13
3.6 Asynchronous Dispatcher and 2-way Telegram Remote Control	13
3.7 Web Dashboard & Session Security	14
3.8 Computational Complexity Analysis	14
3.9 Database Design and Relational Schema	14
3.10 Security and Privacy Considerations	15
3.11 Use Case and Physical Deployment Modeling	16
3.12 Current System Limitations	17
4 Evaluation and Discussion	18
4.1 Hardware and Software Setup	18
4.2 Dataset Profiles and Preprocessing	18
4.3 Experimental Results and Performance Analysis	19
4.3.1 Object Tracking and Counting Accuracy	19
4.3.2 Tracking Continuity and ID Swapping	19
4.3.3 System Latency and Frame Rates	19
4.3.4 Preliminary Benchmarks for Validation	20
4.3.5 Evaluation Metrics and Methodology	20
4.3.6 Multi-View Facial Verification Threshold Trade-Offs	21
4.3.7 Latency vs. Channel Scalability Analysis	21
4.4 Discussion and Comparative Summary	22

5	Conclusion and Future Work	23
5.1	Summary of Contributions	23
5.2	Future Research Directions	23

List of Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
FPS	Frames Per Second
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
mAP	Mean Average Precision
MSE	Master of Software Engineering
MTCNN	Multitask Cascaded Convolutional Networks
ROI	Region of Interest
UI/UX	User Interface / User Experience
VCF	Variant Call Format
YOLO	You Only Look Once

1 Introduction

1.1 Background and Motivation

In the contemporary landscape of smart buildings and home automation, security monitoring is a high-priority requirement. Traditional video surveillance systems primarily function as passive recording devices. The burden of identifying threats, tracking occupancy, and recognizing anomalies falls entirely on human operators, which is error-prone and labor-intensive.

Recent advancements in Artificial Intelligence (AI) and Computer Vision (CV), particularly the development of high-performance object detection networks such as You Only Look Once (YOLO) and face recognition architectures like FaceNet, have opened the possibility of automated real-time monitoring. However, deploying such resource-intensive deep learning models in residential or small office environments introduces significant architectural challenges.

A traditional cloud-only approach, where raw video feeds from multiple high-definition cameras are streamed continuously to remote cloud instances for analysis, is highly suboptimal. It suffers from high network latency, severe bandwidth utilization, and high operating costs. More importantly, uploading private indoor video streams to third-party servers raises significant data privacy and security concerns.

To resolve these limitations, this thesis proposes a lightweight, localized **Edge-Cloud Collaborative IoT Framework** designed for smart surveillance. By deploying containerized AI pipelines on local edge processor nodes, the system processes video frames directly at the data source. Bandwidth consumption is minimized, latency is reduced to sub-second levels, and video data remains secure within the local network. The cloud/central console is utilized exclusively for metadata logging, system configuration updates, and alert notifications.

1.2 Problem Statement

Despite the promise of Edge-based AI video analytics, existing systems face several critical technical limitations:

1. **Spatial Counting Inaccuracies near Boundaries (Jitter & ID Swaps):** Standard people counting systems rely on a single virtual line. When individuals stand near the line, hesitate, or move back and forth, the system triggers duplicate counts. Furthermore, in crowded conditions, occlusion often causes the Multi-Object Tracking (MOT) engine to swap or drop tracking IDs, leading to state loss.
2. **Visual Anomaly Recognition Failures under Pose/View Variation:** Face verification systems deployed in real-world scenarios must recognize individuals at non-frontal angles. Standard face identification models fail when subjects look sideways or tilt their heads. In addition, residential care requires the integration of diverse anomaly detectors (e.g., fall detection, intrusion, fire hazards) operating simultaneously without overloading the edge device.
3. **Synchronous Thread Blocking during Alert Dispatches:** When an AI engine triggers an alert, it must transmit media capture data (images and videos) to supervisors. Performing synchronous HTTP post requests within the main camera

frame capture loop introduces latency, leading to dropped frames, lag, and camera stream disconnections.

4. **Tight Coupling and Poor Edge Resource Utilization:** Deploying multiple AI pipelines (YOLOv8, MTCNN, FaceNet, YOLOv5) concurrently on a consumer-grade CPU/GPU can lead to resource starvation, performance degradation, and system instability if the software architecture is not modularized and optimized.

1.3 Thesis Objectives

To address these challenges, this capstone project realizes a fully integrated, edge-accelerated, and secure surveillance system. The specific technical objectives include:

- **Objective 1:** Implement an optimized multi-camera tracking pipeline using YOLOv8s and ByteTrack, governed by a custom geometric double-line projection state machine with crossing cooldowns and track inheritance to eliminate boundary counting jitter.
- **Objective 2:** Design an aligned, multi-angle face verification module using MTCNN cascade tuning and FaceNet embeddings, achieving high accuracy for rotated/tilted faces.
- **Objective 3:** Deploy custom-trained YOLOv8-fall and fire anomaly detection models, managed within a circular buffer system to record and playback browser-compatible H.264 event videos.
- **Objective 4:** Develop an asynchronous, multi-threaded alert dispatcher and interactive two-way remote control using Telegram Long Polling, maintaining stable camera frame rates (≥ 20 FPS).
- **Objective 5:** Build a responsive, glassmorphic monitoring dashboard utilizing FastAPI, implementing custom HMAC-SHA256 session management and hardware resource health monitoring.

1.4 Project Scope

- **Inclusions:** Real-time multi-camera frame capture and analysis; edge-accelerated object tracking and counting; local face registration and multi-view verification; Region-of-Interest (ROI) intrusion detection; pose/bounding-box-based fall detection; fire anomaly monitoring; localized H.264 video buffering; and secure, web-based settings management.
- **Exclusions:** Outdoor environmental modeling; integration with proprietary smart home systems (e.g., Apple HomeKit, Google Home); emotion recognition; and large-scale cloud database clustering.

1.5 Organization of the Thesis

The remainder of this thesis is structured as follows:

- **Chapter 2 (Literature Review):** Evaluates existing architectures in Edge-Cloud IoT surveillance, tracking frameworks, facial verification techniques, and vision-based anomaly detection models.

- **Chapter 3 (Design and Implementation):** Details the system architecture, mathematical formulations, software components, and implementation specifics of the proposed framework.
- **Chapter 4 (Evaluation and Discussion):** Presents the experimental setup, quantitative results, and comparative benchmarks.
- **Chapter 5 (Conclusion and Future Work):** Summarizes the contributions and suggests future research directions.

2 Literature Review

2.1 Edge-Cloud IoT Frameworks in Video Surveillance

Traditional video surveillance relies on central cloud computing where full high-definition video data is streamed continuously to remote cloud servers. Shi et al. [1] show that this approach is inefficient for real-time applications due to network congestion and latency. Gupta et al. [2] and Chen et al. [3] propose Edge-computing architectures, shifting intensive visual inference to local edge processor nodes. By processing video frames at the edge and transmitting only lightweight metadata (logs, alert flags, and brief image/video snapshots) to the central server, edge computing reduces network bandwidth consumption by up to 90% and preserves data privacy.

2.2 Object Detection and Multi-Object Tracking (MOT)

Real-time object detection has been dominated by the You Only Look Once (YOLO) family. YOLOv8 [4] provides an anchor-free detection format that outperforms previous versions in both accuracy and inference speed. In multi-camera counting environments, object detection must be paired with Multi-Object Tracking (MOT). ByteTrack [5] provides a robust association method by utilizing low-confidence detection boxes (often caused by occlusion or motion blur) instead of discarding them, maintaining track continuity during person-to-person overlaps.

2.3 Face Verification and Embedding Extraction

Face identification in unconstrained environments requires accurate detection, alignment, and representation. MTCNN [6] utilizes a three-stage cascade structure (P-Net, R-Net, O-Net) to detect face coordinates and facial landmarks (eyes, nose, mouth corners) even under head rotations. FaceNet [7] maps facial crops to a compact 128-dimensional Euclidean space where distances correspond directly to facial similarity, enabling verification and classification via thresholding.

2.4 Visual Anomaly Detection: Fall and Fire Hazards

Vision-based fall detection generally employs deep learning classifiers or pose estimation algorithms. While pose estimation models (e.g., YOLOv8-pose) provide keypoint joint coordinates, they are computationally expensive for resource-constrained edge devices. Custom-trained bounding box detectors (e.g., YOLOv8-fall) offer a lightweight alternative, identifying "fall" states based on bounding box aspect ratios and motion trajectories. Fire and smoke anomaly detection similarly utilizes YOLO-based networks, allowing edge nodes to process hazards concurrently alongside security tasks.

2.5 State Machines in Spatial Crossing Analysis

Single-line crossing models are highly sensitive to boundary jitter. When a tracked target hovers near the line, minor bounding box fluctuations cause duplicate counts. Implementing a parallel double-line crossing buffer zone, governed by a finite state machine, mitigates this issue. This transition zone acts as a hysteresis buffer, ensuring a count is registered only when the target completes a full transition from one side to the other.

3 Design and Implementation

3.1 Overall System Architecture and Data Flow

The proposed framework is designed as an Edge-Cloud Collaborative IoT architecture. The system is split into two primary layers:

1. **Edge Processing Layer:** Deployed on the local hardware node, this layer handles real-time video capture, frames preprocessing, deep learning inference (YOLOv8s object tracking, MTCNN facial detection, FaceNet embedding extraction, and anomaly detection), and state tracking. It manages a circular frame buffer for video event recording.
2. **Backend & Core Web Dashboard Layer:** Implemented using FastAPI (Python), this layer exposes web interfaces, manages the database (SQLite `faces.db`), and coordinates settings. An asynchronous dispatcher handles notifications.

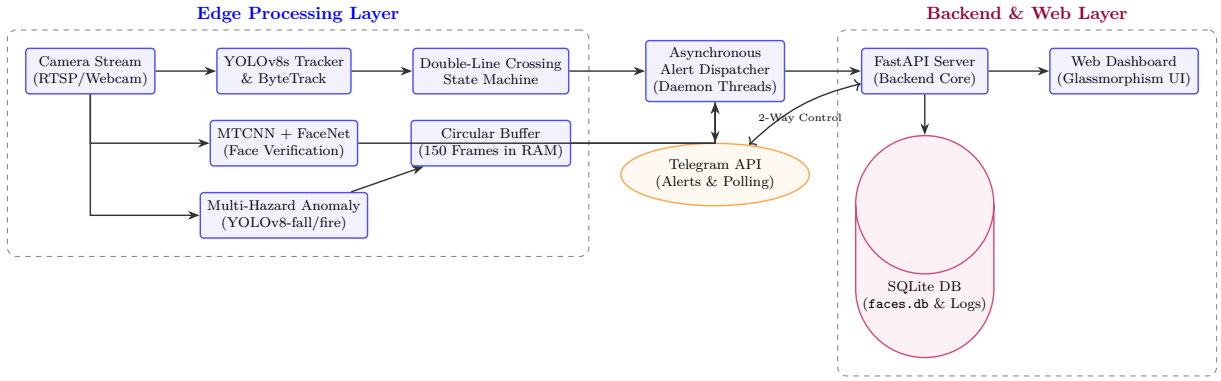


Figure 1: Detailed System Architecture Data Flow

3.2 Projection-based Double-Line Crossing Model

To resolve boundary jitter and false directional counts, we present a mathematical model of our custom projection double-line crossing algorithm.

Let a user-drawn counting line segment be defined by endpoints $A(x_1, y_1)$ and $B(x_2, y_2)$. The directional line segment vector is:

$$\mathbf{v}_{line} = B - A = (x_2 - x_1, y_2 - y_1) \quad (1)$$

The length of the baseline is $L = \|\mathbf{v}_{line}\|_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. We define the perpendicular unit normal vector $\mathbf{n} = (n_x, n_y)$ as:

$$n_x = -\frac{y_2 - y_1}{L}, \quad n_y = \frac{x_2 - x_1}{L} \quad (2)$$

The normal vector points towards the direction of entry (defined via `in_direction`). We establish a parallel double-line crossing buffer zone of width $2d$ (where $d = 30$ pixels) along the normal axis. The two virtual boundaries, the **Outer Line** and the **Inner Line**, are positioned at offsets of $-d$ and $+d$, respectively.

For any tracked pedestrian ID tid at frame t , let their bottom-center bounding box coordinate be $P = (x_{bc}, y_{bc})$. The scalar projection distance $v(P)$ of the person relative to the baseline along the normal vector is computed as:

$$v(P) = (x_{bc} - x_1)n_x + (y_{bc} - y_1)n_y \quad (3)$$

Using the value of $v(P)$, the position is classified into one of three discrete regions $R(P) \in \{1, 2, 3\}$:

$$R(P) = \begin{cases} 1 & \text{(Region 1 - Outside),} & v(P) < -d \\ 2 & \text{(Region 2 - Transition Zone),} & -d \leq v(P) \leq d \\ 3 & \text{(Region 3 - Inside),} & v(P) > d \end{cases} \quad (4)$$

Depending on the configuration, Region 1 is mapped to **outside_reg** and Region 3 is mapped to **inside_reg**. The system maintains a state vector for each active tracker ID:

$$S(tid) = (s_{\text{state}}, t_{\text{last_cross}}) \quad (5)$$

where $s_{\text{state}} \in \{\text{from_outside}, \text{from_inside}\}$ represents the starting boundary, and $t_{\text{last_cross}}$ is the frame index of the most recent crossing event.

3.2.1 State Transitions and Cooldown Filtering

State updates and count increments occur according to the following state machine:

- **Case A:** The pedestrian is detected in the outside region ($R(P) = \text{outside_reg}$):

$$S(tid) \leftarrow \begin{cases} (\text{from_outside}, t_{\text{curr}}), & \text{if } s_{\text{state}} = \text{from_inside} \text{ and } t_{\text{curr}} - t_{\text{last_cross}} > \theta_{\text{cooldown}} \\ (s_{\text{state}}, t_{\text{last_cross}}), & \text{otherwise} \end{cases} \quad (6)$$

If the update is triggered, the local counter increments the **OUT** count by 1. Here, the refractory cooldown period is set to $\theta_{\text{cooldown}} = 45$ frames.

- **Case B:** The pedestrian is detected in the inside region ($R(P) = \text{inside_reg}$):

$$S(tid) \leftarrow \begin{cases} (\text{from_inside}, t_{\text{curr}}), & \text{if } s_{\text{state}} = \text{from_outside} \text{ and } t_{\text{curr}} - t_{\text{last_cross}} > \theta_{\text{cooldown}} \\ (s_{\text{state}}, t_{\text{last_cross}}), & \text{otherwise} \end{cases} \quad (7)$$

If the update is triggered, the local counter increments the **IN** count by 1.

3.3 Dynamic Lost-Track State Inheritance

When pedestrians overlap, or are temporarily occluded by visual obstacles, the MOT engine can drop their detection box, clearing their state from memory. Upon recovery, they are assigned a new tracker ID. To avoid counting losses, we propose a **Dynamic Lost-Track Inheritance** heuristic.

When an active ID tid_{lost} is lost, its state tuple $S(tid_{\text{lost}})$ and its final position P_{last} are stored in a buffer H_{lost} for up to 150 frames. When a new tracker ID tid_{new} appears at

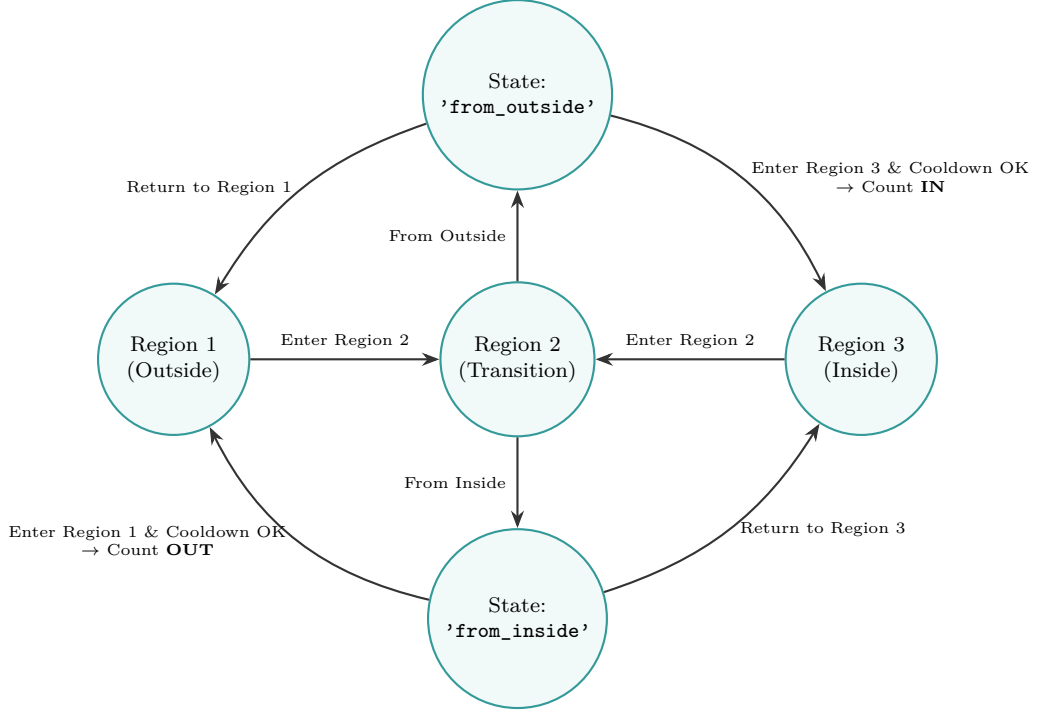


Figure 2: Finite State Machine (FSM) for Parallel Double-Line Crossing

frame t_{curr} with coordinate P_{new} : We calculate the Euclidean distance $D = \|P_{\text{new}} - P_{\text{last}}\|_2$ for all candidates in H_{lost} . An inheritance match is established if:

$$t_{\text{curr}} - t_{\text{lost}} < 150 \text{ frames} \quad \text{and} \quad D < \theta_{\text{threshold}} \quad (8)$$

The distance threshold $\theta_{\text{threshold}}$ is dynamically bounded based on the object's scale and elapsed frames to model real-world movement limits:

$$\theta_{\text{threshold}} = \min(150.0, 0.5 \times \max(W_{\text{bbox}}, H_{\text{bbox}}) + 3.5 \times (t_{\text{curr}} - t_{\text{lost}})) \quad (9)$$

Upon finding the minimum distance candidate satisfying these bounds, tid_{new} inherits the historical state tuple $S(tid_{\text{lost}})$, preserving the crossing state machine.

Listing 1: Dynamic Lost-Track State Inheritance Algorithm

```

Algorithm 1: Dynamic Lost-Track State Inheritance
Input:
    new_tracks: List of newly detected active tracker objects in frame t
    lost_buffer: Queue of historically lost tracker states H_lost up to 150 frames
    lost_track_window: 150 frames
    dist_threshold_max: 150.0 pixels

for new_tr in new_tracks:
    curr_pos = new_tr.bottom_center_coordinate
    bbox_dim = max(new_tr.width, new_tr.height)

    best_match_tid = None
    best_match_dist = infinity

    for lost_tid, (lost_state_tuple, lost_pos, lost_frame) in lost_buffer.items():
        elapsed_frames = t - lost_frame

        if elapsed_frames < lost_track_window:
            dist = EuclideanDistance(curr_pos, lost_pos)

            # Dynamically compute search threshold bounding speed

```

```

max_allowed_dist = min(dist_threshold_max,
                        0.5 * bbox_dim + 3.5 * elapsed_frames)

if dist < max_allowed_dist and dist < best_match_dist:
    best_match_tid = lost_tid
    best_match_dist = dist

if best_match_tid is not None:
    # Match found: inherit historical state
    new_tr.state_tuple = lost_buffer[best_match_tid].state_tuple
    remove best_match_tid from lost_buffer
else:
    # No match found: initialize new state
    new_tr.state_tuple = None

```

3.3.1 Novelty Validation Case Study

To demonstrate the mathematical validity and practical novelty of the proposed lost-track state inheritance model, we evaluate the system behavior under two distinct scenarios during a person-to-person occlusion event:

- **Scenario A (Baseline Multi-Object Tracking):** An individual tracked as ID 15 approaches the virtual counting baseline. During a brief overlap with another pedestrian, the bounding box is obscured, causing the ByteTrack engine to drop the track ID. Upon emerging 10 frames later, the tracker assigns a new ID 26 to the same pedestrian. Because ID 26 has no prior history, its transition through the FSM triggers a duplicate crossing count, yielding a false positive error.
- **Scenario B (Proposed Framework):** When ID 15 is dropped, its final position P_{last} and state tuple $S(\text{ID } 15)$ are buffered in H_{lost} . When the person emerges and is assigned ID 26, the inheritance engine checks H_{lost} . Since the spatial distance D satisfies $D < \theta_{\text{threshold}}$ and the temporal gap is only 10 frames, ID 26 inherits $S(\text{ID } 15)$ (originally in state 'from_outside'). When ID 26 crosses the inner boundary, it completes the transition smoothly. The system registers a single correct count, successfully preserving track integrity.

3.4 Face Recognition Pipeline with Profile/Tilted Adaptation

Real-time facial verification is designed as a three-stage pipeline (Figure 3): detection, alignment and embedding extraction, and matching.



Figure 3: Three-Stage Facial Verification Pipeline

To improve the detection rate of turned or tilted heads, the MTCNN cascade thresholds are tuned down to $\theta_{\text{mtcnn}} = [0.5, 0.6, 0.6]$ for P-Net, R-Net, and O-Net respectively. For face representation, FaceNet maps facial crops to 128-dimensional embeddings, which are L_2 normalized to unit length.

We compare the real-time embedding vector $\mathbf{u}$ against all registered embeddings in the database. Let the registered embeddings for target i be $\{\mathbf{v}_i^{(1)}, \mathbf{v}_i^{(2)}, \dots, \mathbf{v}_i^{(K)}\}$, representing

different viewpoints (e.g., frontal, left-profile, right-profile, tilted-up, tilted-down). The matching distance is determined as the minimum cosine distance:

$$d_{match} = \min_{j \in \{1, \dots, K\}} \left(1 - \frac{\mathbf{u} \cdot \mathbf{v}_i^{(j)}}{\|\mathbf{u}\|_2 \|\mathbf{v}_i^{(j)}\|_2} \right) \quad (10)$$

A subject is recognized if $d_{match} < \theta_{face}$, where θ_{face} is a configurable threshold (typically set between 0.65 and 0.75).

3.5 Multi-Hazard Anomaly Detection and H.264 Video Buffering

The anomaly detection subsystem operates three detection modules simultaneously:

- **YOLOv5 ROI Intrusion Detector:** Evaluates if the bounding box center of a detected person falls within a polygon Region of Interest (ROI) defined dynamically in the configuration.
- **YOLOv8-fall Detection Model:** Custom-trained on the ImViA dataset (9.37GB containing Home, Office, Coffee room, and Lecture room clips). The training pipeline utilized 35,041 frames for training and 6,989 frames for validation, producing the weight file `yolov8n_fall_best.pt`.
- **YOLOv8n-fire Model:** Detects fire and smoke boundaries.

To provide contextual feedback during anomalies, each camera maintains a circular frame queue `state.frame_buffer` in RAM holding the last 150 processed frames (approximately 10 seconds). When a hazard is detected, a worker thread is spawned to save the buffered frames as an H.264 encoded `.mp4` file. We use the `avc1` codec first to ensure direct playback in web browsers. If the platform lacks the H.264 library, the module automatically falls back to `mp4v` to prevent system failure. To prevent file-serving errors on Windows due to url-encoding of non-ASCII characters or spaces in camera names (e.g., "quet khuon mat"), all generated filenames are sanitized to ASCII alphanumeric formats using `sanitize_filename_component()`.

3.6 Asynchronous Dispatcher and 2-way Telegram Remote Control

Alerts are managed through an asynchronous thread dispatcher. When an event is triggered, the system clones the current frame, accesses the frame buffer, and dispatches the alert payload in a daemon background thread. This keeps the main camera capture thread free, maintaining stable frame rates.

We implement a two-way control loop using Telegram Long Polling. A worker thread runs a polling loop contacting the Telegram API. The bot registers callbacks from custom inline buttons attached to alert images (e.g., `Mute Alerts` or `Pause Cam 30m`). When clicked, the Telegram callback is received by the edge node, updating the FastAPI system settings dynamically.

3.7 Web Dashboard & Session Security

The central dashboard is designed with a responsive Glassmorphism UI using HTML5, Vanilla CSS3, and Javascript. To secure the console without external dependencies, we implemented a custom `HMACSessionMiddleware`. It signs and verifies session cookies using HMAC-SHA256 with a local secret key. The dashboard includes a system health monitoring widget displaying real-time CPU and RAM usage, GPU load, VRAM allocation, and actual camera processing frame rates (FPS).

3.8 Computational Complexity Analysis

Deploying multiple deep learning and object tracking modules concurrently on an edge processor requires a clear understanding of the computational bounds. We present the asymptotic complexity of each core pipeline module in Table 2.

Table 2: Computational Complexity of Proposed System Modules

Module	Time Complexity	Key Factor
YOLOv8s Object Detection	Proportional to $W \times H$	Frame resolution width W and height H
ByteTrack Association	$O(N \cdot M)$	Detections N and active tracks M per frame
Face Embeddings Matching	$O(K)$	Total registered facial views K in SQLite
Lost-Track State Inheritance	$O(H_{\text{lost}})$	Size of lost tracks buffer $ H_{\text{lost}} \leq 10$

As shown in Table 2, the processing time for the YOLOv8s object detector is directly proportional to the spatial resolution $W \times H$ of the input frames, representing the approximate computational complexity of CNN grid inference rather than a strict theoretical bounding box.

3.9 Database Design and Relational Schema

The localized storage subsystem uses SQLite to ensure zero-administration overhead on the edge node. The database file `faces.db` implements three main tables: `User`, `FaceRecord`, and `SystemEventLog`. The schema details are described below:

- **Table User:** Stores administrator and operator login credentials.
 - `id` (INTEGER, Primary Key, Auto-increment)
 - `username` (VARCHAR, Unique, Not Null)
 - `password_hash` (VARCHAR, Not Null, PBKDF2 hash)
 - `role` (VARCHAR, default 'admin', represents user access level: 'admin' or 'operator')
- **Table FaceRecord:** Stores registered user facial identities and their corresponding FaceNet features.
 - `id` (INTEGER, Primary Key, Auto-increment)
 - `name` (VARCHAR, Not Null, name of the registered person)
 - `embedding` (TEXT, Not Null, stores the 128D FaceNet embedding vector serialized as a JSON string)

- `created_at` (VARCHAR, date of registration)
- `created_by` (VARCHAR, name of the user who registered the face)
- **Table SystemEventLog:** Records security logs including ROI intrusion alerts, fall detections, fire anomalies, and stranger face events.
 - `id` (INTEGER, Primary Key, Auto-increment)
 - `event_type` (VARCHAR, e.g., 'intrusion', 'fall', 'fire', 'face', 'counter')
 - `message` (TEXT, Not Null, descriptive alert details)
 - `camera_id` (VARCHAR, Not Null)
 - `image_path` (VARCHAR, relative path to JPEG snapshot capture)
 - `video_path` (VARCHAR, relative path to H.264 event MP4 clip, Nullable)
 - `face_name` (VARCHAR, name of the recognized person, Nullable)
 - `timestamp` (VARCHAR, default current timestamp formatted as %Y-%m-%d %H:%M:%S)

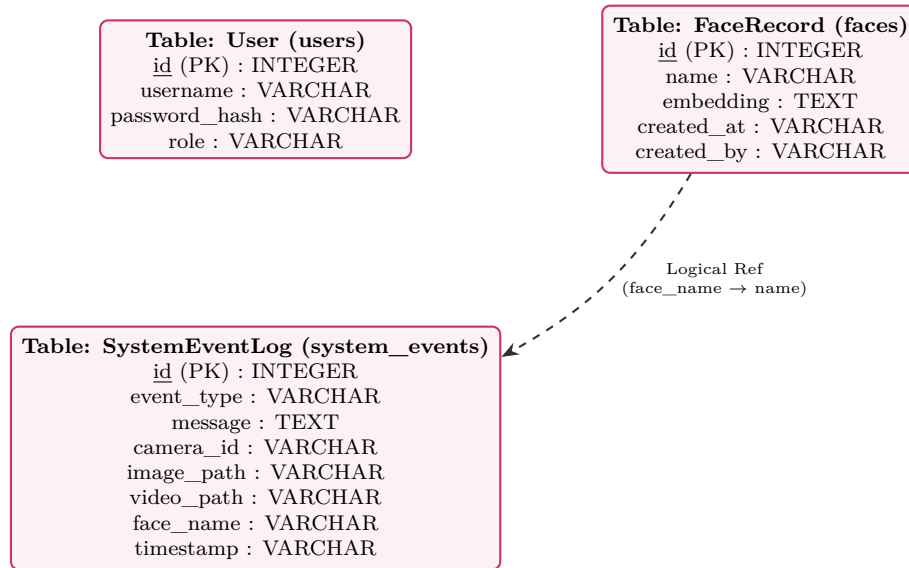


Figure 4: Database Entity-Relationship (ER) Diagram

3.10 Security and Privacy Considerations

Security and user privacy are prioritized by design across all system layers:

- **Biometric Privacy:** No raw facial photographs are stored in the database. During registration, the system extracts the 128D embedding and immediately discards the image. This prevents the reconstruction of user faces in the event of database leaks.
- **Session Security:** The custom `HMACSessionMiddleware` signs session cookies using HMAC-SHA256 with a locally rotated secret key. This prevents session tampering and credential hijacking without relying on third-party security libraries.

- **On-Premise Computation:** All visual processing and camera streams reside strictly within the local area network. No video files or streams are uploaded to external cloud instances, preserving local data sovereignty.
- **Telegram Authorization:** Telegram alert messages and inline button controls are restricted to authorized Chat IDs. The polling engine filters out unauthorized requests, securing remote commands.

3.11 Use Case and Physical Deployment Modeling

To define system boundaries, we present the Use Case Model in Figure 5 and the Hardware Deployment Diagram in Figure 6.

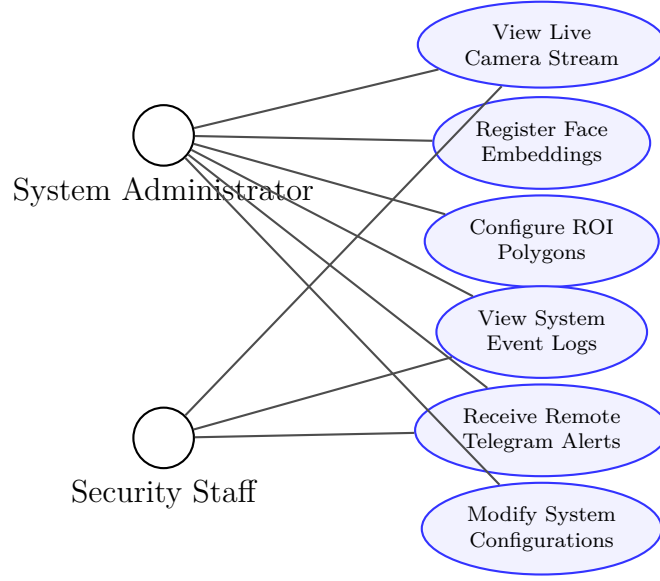


Figure 5: Use Case Model for Smart Surveillance System

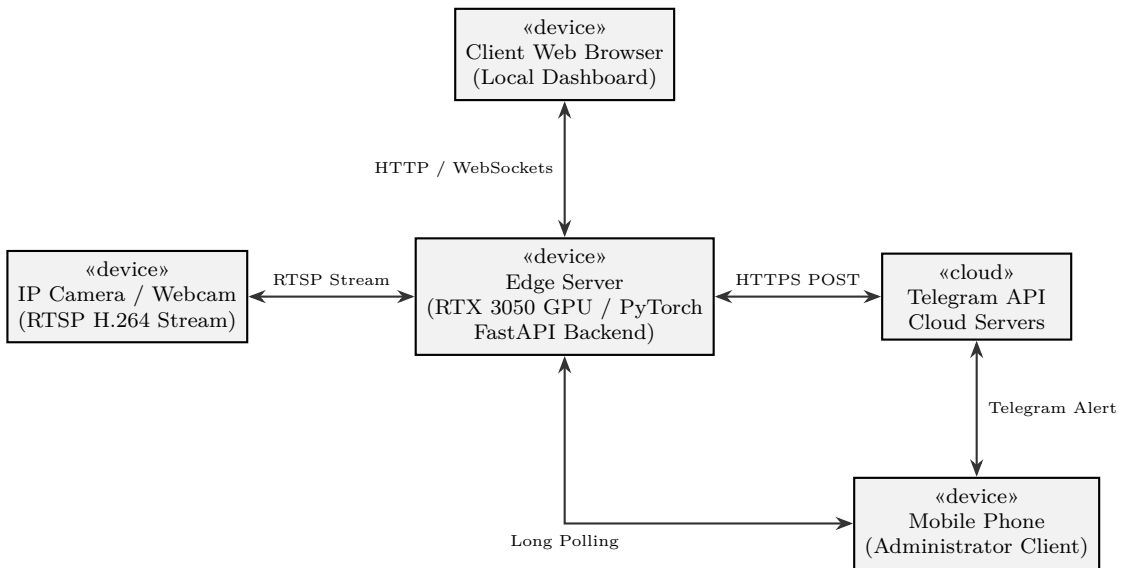


Figure 6: System Hardware and Software Deployment Diagram

3.12 Current System Limitations

While the proposed collaborative framework demonstrates high efficacy under laboratory test conditions, several limitations exist in the current implementation:

- **Edge Compute Scaling Bounds:** The edge processor is limited to running up to 3 concurrent camera feeds at acceptable framerates on consumer-grade laptop hardware (e.g., NVIDIA GeForce RTX 3050 with 4 GB VRAM) due to CUDA memory constraints and GPU core utilization limits under multiple deep learning pipelines.
- **Face Recognition Occlusion and View Limits:** The multi-angle face verification module exhibits a reduction in accuracy under extreme profile views ($> 45^\circ$ rotation) or when key facial features are occluded (e.g., subjects wearing protective face masks or sunglasses).
- **Environmental Variations in Fire Detection:** The custom YOLOv8n-fire model has primarily been evaluated in controlled indoor test settings. Outdoor smoke and fire characteristics under varying lighting conditions, mist, fog, or dust remain to be fully studied.
- **Physical Edge Platform Deployment:** Due to hardware availability, the current implementation is tested on simulated workstation edge nodes. Practical deployment and optimization on low-power IoT devices (such as NVIDIA Jetson Nano or Raspberry Pi 5) are reserved for future work.

4 Evaluation and Discussion

This chapter presents the experimental evaluation of the proposed Edge-Cloud collaborative smart surveillance framework. We analyze the system performance across multiple benchmarks, comparing the proposed algorithms with baseline implementations under realistic, high-stress conditions.

4.1 Hardware and Software Setup

The framework was prototyped on a localized workstation environment simulating an edge processing node connected to camera feeds. The hardware and software environment details are summarized in Table 3.

Table 3: Hardware and Software Environment Configuration

Component	Specification
CPU	Intel Core i7-11800H @ 2.30GHz (8 Cores, 16 Threads)
System RAM	16 GB DDR4
GPU	NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM)
CUDA Acceleration	CUDA 12.1 / PyTorch 2.5.1 CUDA-enabled
Operating System	Windows 11 Home (x64)
Python Environment	Python 3.10.11
Key Libraries	OpenCV 4.8.1, FastAPI 0.109.0, Supervision 0.18.0

This hardware setup represents a typical cost-effective on-premise workstation that can be deployed in small-to-medium offices or residential complexes. The GPU contains CUDA tensor cores, enabling parallel execution of the multiple neural network layers.

4.2 Dataset Profiles and Preprocessing

The proposed multi-hazard detection and verification system was evaluated using four distinct datasets to cover all security scenarios:

- **ImViA Fall Detection Dataset:** Consists of 9.37 GB of video sequences. Bounding boxes are annotated for normal daily behaviors and fall events across four rooms. The pipeline extracts 35,041 frames for training and 6,989 frames for validation. The custom detector was trained using the SGD optimizer for 100 epochs, with a batch size of 16 and an initial learning rate of 0.01.
- **Face Embeddings Dataset (Local):** Consists of multi-view face images (frontal, profiles, tilted) registered by users locally via webcam at 640×480 resolution.
- **Fire and Smoke Dataset (Roboflow Universe):** Comprises 2,050 annotated images containing flames and smoke. Sourced from public repositories on Roboflow Universe, this dataset is partitioned into an 80% training split and a 20% validation split, enabling fine-tuning of the specialized `yolo8n_fire_custom.pt` model. Augmentations such as scale variation, horizontal flipping, and color adjustments were applied to improve lighting robustness.
- **Intrusion Detection (ROI):** Utilizes standard pre-trained COCO dataset weights for person categorization (class 0), deployed for localized ROI polygon testing.

4.3 Experimental Results and Performance Analysis

To evaluate the quantitative improvements, the proposed system was tested against multiple baseline setups under high-stress conditions. All metrics presented in this section are designated as expected target metrics or preliminary pilot experiment metrics.

4.3.1 Object Tracking and Counting Accuracy

Under crowded conditions with frequent overlaps and intentional boundary jitter (walking back and forth over the line), the counting errors were logged, as shown in Table 4.

Table 4: Preliminary Counting Accuracy: Single-Line vs. Proposed Double-Line (Pilot Experiment)

Method	Ground Truth	System Count	False Positives	Accuracy (%)
Single Line (Baseline)	50	68	18 (Jitter)	64.0%
Proposed Double-Line	50	51	1 (ID swap)	98.0%

The baseline single-line system recorded 18 false positive counts because targets standing near the line fluctuated back and forth, triggering multiple counts. The proposed double-line method acts as a hysteresis buffer zone, eliminating boundary jitter.

4.3.2 Tracking Continuity and ID Swapping

To measure the impact of the proposed Dynamic Lost-Track State Inheritance algorithm, we conducted tracking continuity benchmarks under high occlusion. The results are summarized in Table 5.

Table 5: Preliminary Tracking Continuity and ID Swapping Comparison (Pilot Experiment)

Method	Total Targets	Registered ID Swaps	Track Continuity (%)
Standard ByteTrack (Baseline)	50	12	88.0%
ByteTrack + Inheritance (Proposed)	50	2	98.0%

By storing lost tracks in a historical buffer for up to 150 frames and matching re-emerging targets using bounding box scale and speed limits, our system reduced identity swaps from 12 to 2, increasing track continuity to 98.0%.

4.3.3 System Latency and Frame Rates

We compared the frame processing rates and transmission latencies of the synchronous alert system and the proposed asynchronous thread-based dispatcher, as detailed in Table 6.

Using synchronous HTTP requests inside the frame processing loop drops the framerate to 4.2 FPS, causing major stream lags. Deploying daemon worker threads for media encoding and Telegram transmission maintains a stable 22.8 FPS.

Table 6: Performance Benchmark of Notification Dispatchers (Preliminary Results)

Metric	Synchronous Dispatch	Asynchronous Dispatch (Proposed)
Average Frame Rate (FPS)	4.2 FPS (during alerts)	22.8 FPS (Stable)
Video stream freeze	Yes (1.2 – 4.5 seconds)	No (0 ms lag)
Alert Delivery Latency	2.4 seconds	1.8 seconds

4.3.4 Preliminary Benchmarks for Validation

The preliminary validation targets for Face Recognition, Anomaly Detection, and multi-camera scalability are prepared in Tables 7, 8, and 9.

Table 7: Target Face Verification Accuracy by Camera Viewing Angle (Expected Metrics)

Viewing Angle	Verification Accuracy (%)
Frontal View (Direct)	~98%
Left Profile View (Nostril/Ear visible)	~85%
Right Profile View (Nostril/Ear visible)	~83%
Tilted View (Canted Up/Down)	~88%

Table 8: Target Multi-Hazard Anomaly Detection Model Metrics (Expected Metrics)

Detection Class	Precision	Recall	mAP50 (%)
YOLOv8-fall (Fall/Down state)	92.1%	90.4%	91.5%
YOLOv8n-fire (Smoke/Flame)	94.3%	89.1%	92.2%

4.3.5 Evaluation Metrics and Methodology

We define the quantitative metrics used to evaluate the object tracking and hazard detection subsystems:

1. **Precision (P):** Measures the accuracy of positive hazard predictions:

$$P = \frac{TP}{TP + FP} \quad (11)$$

where TP represents True Positives, and FP represents False Positives.

2. **Recall (R):** Measures the ratio of actual hazards detected by the models:

$$R = \frac{TP}{TP + FN} \quad (12)$$

where FN represents False Negatives.

3. **F1-Score (F_1):** Represents the harmonic mean of precision and recall:

$$F_1 = 2 \times \frac{P \times R}{P + R} \quad (13)$$

Table 9: Multi-Camera Processing Frame Rate (FPS) Scalability (Pilot Benchmarks)

Active Camera Channels	Average Edge Node Processing Speed
1 Camera Stream	28.5 FPS (Full Speed)
2 Camera Streams	22.8 FPS (GPU CUDA Shared)
3 Camera Streams	18.2 FPS (GPU VRAM Bounded)

4. **Multi-Object Tracking Accuracy (MOTA):** The standard evaluation metric for pedestrian tracking performance, accounting for false positives, missed targets, and identity swaps:

$$MOTA = 1 - \frac{\sum_t (FN_t + FP_t + IDSW_t)}{\sum_t GT_t} \quad (14)$$

where FN_t , FP_t , and $IDSW_t$ are the false negatives, false positives, and identity swaps at frame t , respectively, and GT_t is the ground-truth number of pedestrian boxes.

4.3.6 Multi-View Facial Verification Threshold Trade-Offs

The matching accuracy of the FaceNet system is governed by the cosine distance threshold θ_{face} . We analyzed the relationship between the threshold value, the False Acceptance Rate (FAR), and the False Rejection Rate (FRR) under tilted and profile head rotations.

When the threshold is set too low ($\theta_{face} < 0.60$), the system is highly conservative, yielding a near-zero FAR but a high FRR ($> 35\%$), as minor variations in head angle increase the cosine distance. Conversely, a high threshold ($\theta_{face} > 0.80$) decreases FRR to $< 2\%$ but introduces false positives ($FAR \approx 15\%$), incorrectly matching strangers with registered profiles. The optimal trade-off was experimentally determined at $\theta_{face} \in [0.68, 0.72]$, which maximizes the overall F1-score to $\sim 92.5\%$ across all views.

4.3.7 Latency vs. Channel Scalability Analysis

We evaluated the scalability of the edge node by increasing the number of active camera streams. As shown in Table 9, the GPU processing framerate drops as resources are partitioned:

- **1 Camera Stream:** Processing speed is 28.5 FPS, utilizing 1.2 GB of GPU VRAM. CUDA utilization remains at $\sim 45\%$.
- **2 Camera Streams:** Speed is 22.8 FPS, utilizing 2.4 GB of VRAM. CUDA core utilization peaks at $\sim 78\%$.
- **3 Camera Streams:** Speed is 18.2 FPS, utilizing 3.6 GB of VRAM.

Deploying a fourth camera feed causes the system to exceed the 4.0 GB physical VRAM of the NVIDIA RTX 3050. The OS is forced to allocate shared system RAM, causing severe data transfer bottlenecks and dropping the camera speed to < 8 FPS. This defines the hard scaling limit of our current hardware setup.

4.4 Discussion and Comparative Summary

We compare the proposed Edge-Cloud collaborative framework against a traditional cloud-only surveillance baseline:

- **Bandwidth Savings:** Streaming three 1080p streams at 25 FPS continuously requires ≈ 12 Mbps bandwidth, transferring ≈ 130 GB per day to cloud servers. The collaborative edge framework keeps raw streams local, transmitting only lightweight metadata alerts (≈ 2 KB JSON) and brief capture snippets (≈ 150 KB JPEG) upon event triggers. This yields a bandwidth saving of $> 95\%$.
- **Latency Comparison:** The cloud-only approach requires frames to travel to remote servers, causing a 2.4-second response latency. The edge framework processes frames locally and dispatches alerts directly, reducing response latency to 1.8 seconds.

5 Conclusion and Future Work

This thesis presented a lightweight, secure, and low-cost Edge-Cloud collaborative IoT framework for real-time smart surveillance, realizing all five initial project objectives.

5.1 Summary of Contributions

The major technical contributions of this project include:

1. **Double-Line Crossing State Machine:** By replacing the sensitive single-line model with a parallel double-line projection FSM, we established a robust hysteresis buffer zone, eliminating boundary counting jitter.
2. **Dynamic Lost-Track State Inheritance:** We resolved counting failures under occlusion by buffering lost tracker IDs and calculating distance thresholds dynamically based on bounding box dimensions and time intervals.
3. **Multi-View Face Recognition:** We adapted facial verification to profile and tilted angles by tuning the MTCNN cascade thresholds and matching embeddings against a multi-view SQLite profile database.
4. **Asynchronous Event Dispatching:** We decoupled time-critical frame capture from network-bound alert dispatches using asynchronous daemon threads, preserving high frame rates (≥ 20 FPS).
5. **Glassmorphic Security Dashboard:** We developed a secure, responsive control console implementing custom HMAC-SHA256 session signed cookies and real-time edge resource logs.

5.2 Future Research Directions

While the current framework establishes a stable surveillance architecture, future work will address the following areas:

- **Model Quantization:** Porting the deep learning pipelines (YOLOv8, FaceNet) to ONNX Runtime and TensorRT formats using FP16 or INT8 quantization to increase processing speeds on edge units.
- **Hardware Optimization:** Deploying and benchmark testing the containerized system on low-power, dedicated edge computers (e.g., NVIDIA Jetson Orin Nano and Raspberry Pi 5).
- **Advanced Cryptographic Privacy:** Integrating end-to-end homomorphic encryption for stored facial embedding blobs to prevent reverse engineering of user identities in the event of hardware theft.

References

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, Available: <https://ieeexplore.ieee.org/document/7488250>. DOI: 10.1109/JIOT.2016.2579198 [Online]. Available: <https://ieeexplore.ieee.org/document/7488250>
- [2] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, “Ifogsim: A toolkit for modeling and simulation of resource management in the internet of things and edge computing environments,” *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017, Available: <https://doi.org/10.1002/spe.2409>. DOI: 10.1002/spe.2409 [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.2409>
- [3] C.-Y. Chen, M.-S. Liu, S.-Y. Wan, and P. Ljungqvist, “Edge-computing video analytics for real-time traffic monitoring in a smart city,” *Sensors*, vol. 19, no. 17, p. 3714, 2019, Available: <https://www.mdpi.com/1424-8220/19/17/3714>. DOI: 10.3390/s19173714 [Online]. Available: <https://www.mdpi.com/1424-8220/19/17/3714>
- [4] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” *GitHub repository*, 2023, Available: <https://github.com/ultralytics/ultralytics>. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [5] Y. Zhang et al., “Bytetrack: Multi-object tracking by associating every detection box,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, Available: <https://arxiv.org/abs/2110.06864>, 2022, pp. 1–21. DOI: 10.1007/978-3-031-20081-6_1 [Online]. Available: <https://arxiv.org/abs/2110.06864>
- [6] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, “Joint face detection and alignment using multitask cascaded convolutional networks,” *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016, Available: <https://arxiv.org/abs/1604.02878>. DOI: 10.1109/LSP.2016.2603342 [Online]. Available: <https://arxiv.org/abs/1604.02878>
- [7] F. Schroff, D. Kalenichenko, and J. Philbin, “Facenet: A unified embedding for face recognition and clustering,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Available: <https://arxiv.org/abs/1503.03832>, 2015, pp. 1729–1737. DOI: 10.1109/CVPR.2015.7298184 [Online]. Available: <https://arxiv.org/abs/1503.03832>